ETHEL ADWOA SAKYI - CSCI 390

JAVASCRIPT MEETS POST-QUANTUM CRYPTOGRAPHY

THE ROLE OF JAVASCRIPT IN A QUANTUM WORLD

TABLE OF CONTENT

01. Intro to Post-Quantum Cryptography

02. Why JavaScript Matters in PQC

03. Overview of PQC Algorithms

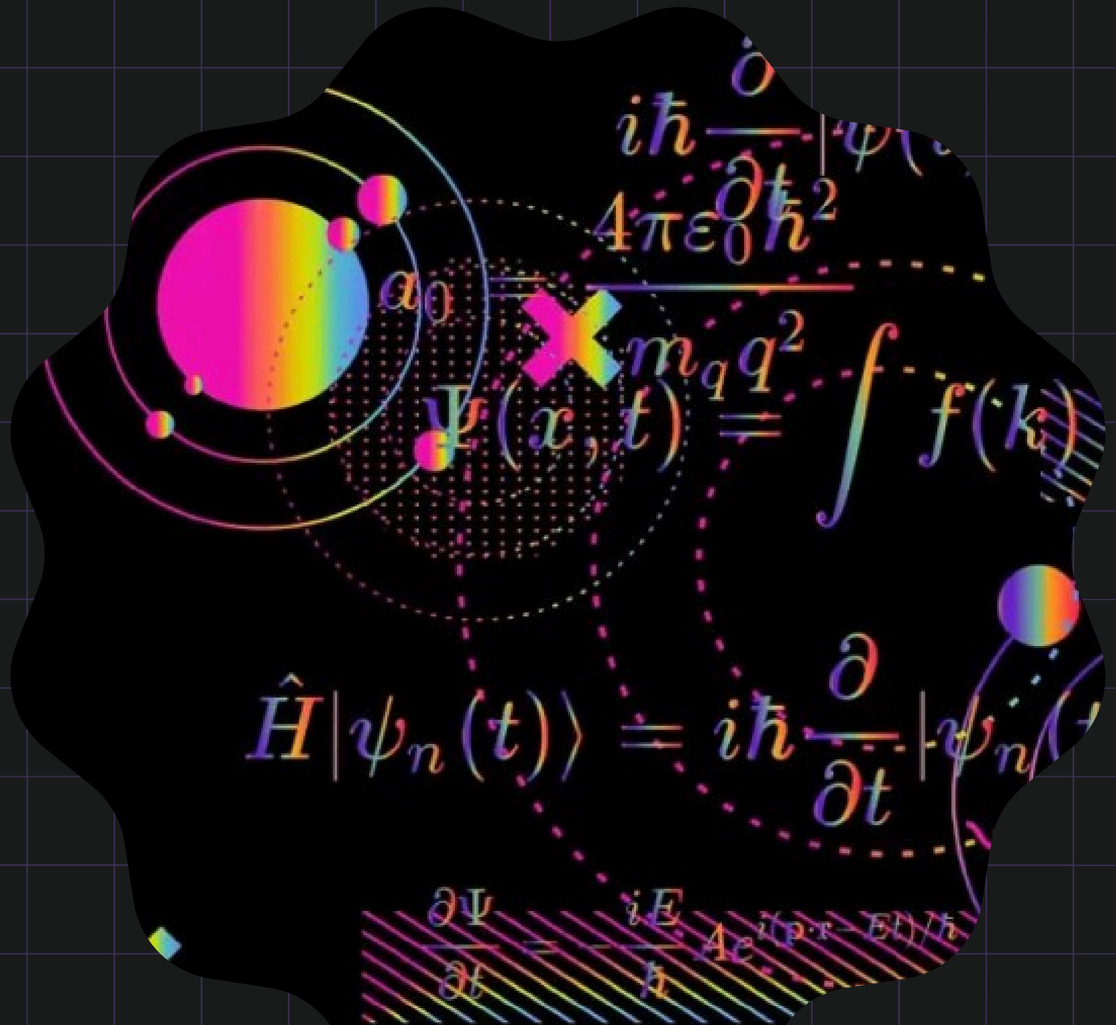
04. Challenges of PQC in JavaScript

06. How JavaScript can Implement PQC

07. Real World Applications

08. The future of JavaScript and PQC

09. Conclusion



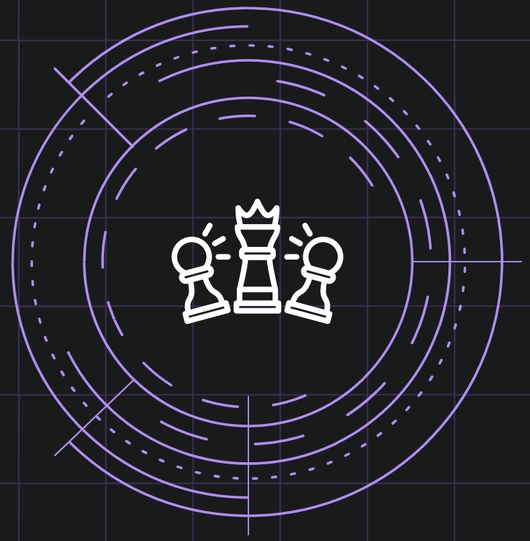
WHAT IS POST-QUANTUM CRYPTOGRAPHY

Quantum Computers can break the current traditional encryption systems - like the RSA (Rivest-Shamir-Adleman) and the ECC (Elliptic Curve Cryptography). These traditional encryption methods are based on NP-hard problems like integer factorization and discrete logarithms. Classical algorithms to solve these problems scale poorly, requiring exponential time for large inputs. The inability for classical computers to properly solve these problems have made them a great option for data encryption and security.

Quantum computers leverage superposition and entaglement to explore many possible solutions simultaneously, making these problems easy to solve. Since quantum computers can break traditional encryption, researchers are developing post-quantum cryptographic algorithms that rely on mathematical problems not susceptible to known quantum attacks.

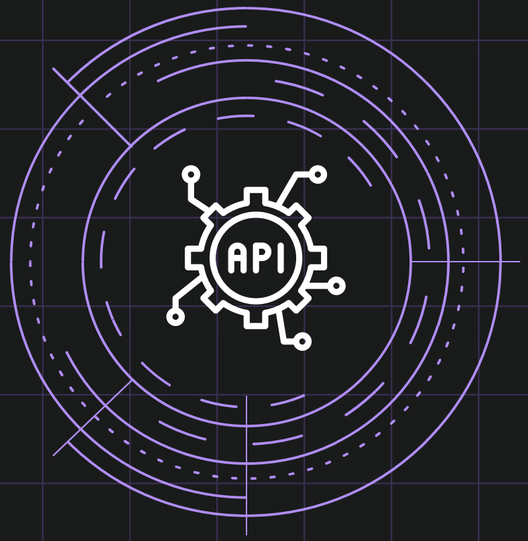


WHY JAVASCRIPT MATTERS



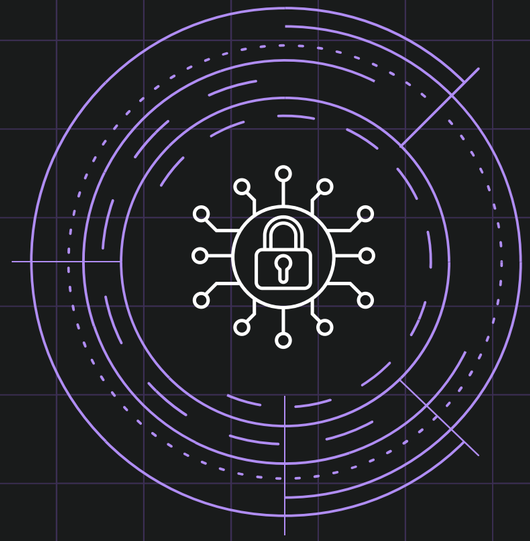
Web-Domination

JavaScript is used in browsers.
It powers client side applications and handles cryptography



Web Cryptography API

JavaScript already supports classical cryptographic operations like RSA and AES



Future of Secure Web Apps

JavaScript will integrate PQC to secure real-time web apps, IoT's and financial systems

PQC ALGORITHMS OVERVIEW

Kyber

Designed for secure key exchange and replaces traditional methods like Diffie-Hellman and RSA.

Relevance for JS

Could be implemented in JavaScript for securing web sessions (HTTPS connections)

SPHINCS+

Relies solely on hash functions and uses Merkle Trees and Winternitz One-Time signatures to create quantum resistant signatures

Relevance for JS

Suitable for use cases where digital signatures are critical, such as signing browser extensions or securing blockchain transactions

CHALLENGES OF PQC IN JS



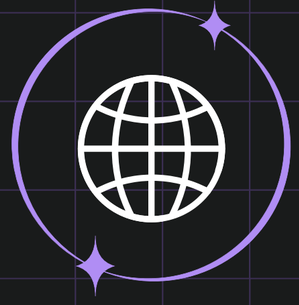
Performance

PCQ Algorithms require more computation, and JavaScript is an interpreted Language. Solution: Use WebAssembly for native like performance.



Key and Signature Size

PQC keys and signatures are larger than traditional ones, impacting both bandwith and memory. JavaScript apps will need optimized handling of these larger data sizes.



Browser Adoption

PCQ needs to also be integrated into the Web Cryptography API.

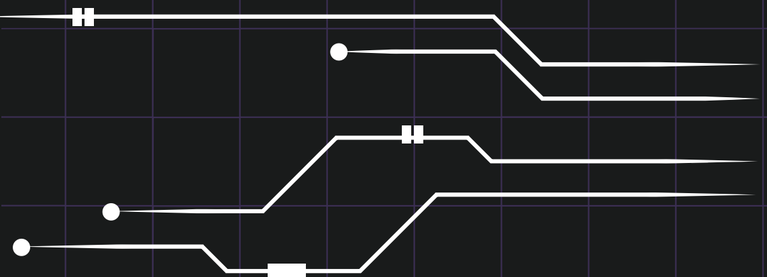
HOW JS CAN IMPLEMENT PQC

```
// Import a library that provides post-quantum cryptographic (PQC) key exchange algorithms.
import { pqKeyExchange } from 'pqc-library';

// Use an asynchronous function to handle the asynchronous operations involved in cryptographic computations
(async function() {
    // Generate a pair of keys (public and private) using the PQC key exchange algorithm.
    const { publicKey, privateKey } = await pqKeyExchange.generateKeys();

    // Compute the shared secret key using your public key and the server's public key.
    // This step establishes a secure channel between the client and server.
    const sharedKey = await pqKeyExchange.computeSharedKey(publicKey, serverPublicKey);

    // Log the shared key to the console for demonstration purposes (not recommended in real applications).
    // The shared key will be the same for both the client and server, ensuring secure communication.
    console.log("Shared key established:", sharedKey);
})();
```



This example demonstrates how JavaScript can implement post-quantum key exchange using a hypothetical PQC library

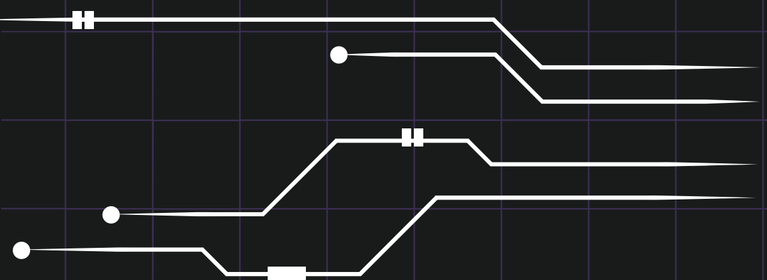
HOW JS CAN IMPLEMENT PQC

```
// Import the WebAssembly module that implements the PQC algorithms.
// The './pqc_wasm.js' file is a wrapper that loads and initializes the Wasm binary.
import initPqcWasm from './pqc_wasm.js';

// Use an asynchronous function to handle loading and interacting with the Wasm module.
(async function() {
    // Initialize the Wasm module and retrieve its API for cryptographic operations.
    const pqc = await initPqcWasm();

    // Use the Wasm-provided API to generate a pair of keys (public and private).
    const keyPair = pqc.generate_keys();

    // Log the generated public key to the console for demonstration purposes.
    console.log("Public Key:", keyPair.publicKey);
})();
```



This example demonstrates how to optimize post-quantum cryptographic computations by using WebAssembly (Wasm) for better performance

REAL WORLD APPLICATIONS

WEB SECURITY

HTTPS with post- quantum TLS (transport layer security). Secures sensitive user data against future quantum attacks

BLOCKCHAIN/WEB3

PCQ can secure JavaScript powered decentralized apps and cryptocurrencies which currently use elliptic curve cryptography for public-private key pairs

IoT DEVICES

Many IoT devices run JavaScript (Node.js). PQC ensures that their communication remains secure.

THE FUTURE OF JS AND PQC

BROWSER INTEGRATION

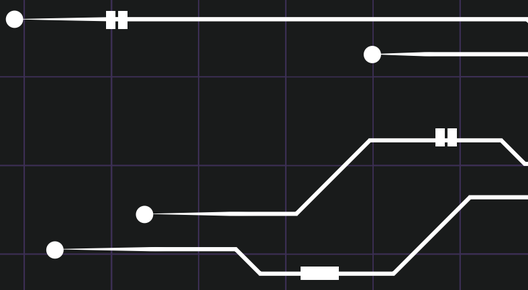
Web Cryptography API will incorporate PQC algorithms, making JavaScript a natural tool for secure post-quantum applications

HYBRID CRYPTOGRAPHY

Transition phase with both classical and post-quantum algorithms could be implemented in JavaScript

GLOBAL IMPACT

JavaScripts role in PQC ensures a secure internet accesible to everyone, this will bridge cryptography and user-friendly applications



CONCLUSION



JavaScript will play a key role in the transition to post-quantum cryptography. Its flexibility, widespread use, and integration with technologies like WebAssembly make it a key tool for securing the future of the web.

ETHEL SAKYI

THANK YOU!

Ethel A Sakyi
CSCI 390